

Helix -- Implementation Report

Project: Helix (Intelligent SDLC Copilot)

Event: Code-AI-Thon 2026, Phase 2 (Prototype)

Repository: <https://github.com/siddham04/AIThon>

Document version: 1.0 (May 2026)

Companion PDF: Prebuilt under docs/pdf/Helix-Implementation-Report.pdf. Regenerate with `pip install -r scripts/requirements-docs-pdf.txt` then `python scripts/build_submission_pdfs.py`.

1. Executive summary

Helix is an AI-assisted platform that turns unstructured requirements (text, files, URLs, optional browser voice-to-text) into a traceable set of engineering artifacts: summaries, user stories, tasks with acceptance criteria, automated test scenarios, ambiguity findings, risk notes, and effort estimates. A React workspace lets teams review, refine, query via a conversational copilot, and export to CSV/Markdown and (when configured) Jira or GitHub.

The system is implemented as a FastAPI backend with PostgreSQL (or SQLite for single-container demos), optional Redis for task progress, optional MongoDB for version snapshots and chunk storage, in-process RAG (FAISS + embeddings), and a Vite + React frontend. Multi-stage generation runs through an orchestrator with Server-Sent Events for live progress; Azure OpenAI and/or Anthropic APIs are used when keys are present, with a deterministic mock/demo path when they are not.

2. Problem statement and goals

Software teams receive requirements from documents, email, meetings, and ad-hoc notes. Manual translation into backlog items, tests, and shared understanding is slow and error-prone. The hackathon goal was to build a platform that ingests raw input, structures it into SDLC artifacts, generates tests, surfaces ambiguity, and provides an interactive dashboard -- plus optional conversational assistance, estimation, and PM-tool export.

Helix addresses each of these explicitly (see Section 4).

3. Scope delivered

- Ingestion: REST endpoints for text, file upload, and URL extraction; preprocessing (cleaning, clause split, optional Mongo chunk storage, sensitive-pattern hints, embeddings for RAG).
 - Structured outputs: User stories, tasks, acceptance criteria, requirement summary, risks, productivity metrics (incl. citation rate and timings).
 - Test generation: Test architect agent producing Given/When/Then style cases linked to stories and clauses.
 - Ambiguity: LLM-based ambiguity agent plus optional NLP heuristics (e.g. passive voice, vague tokens).
 - Dashboard: Full workspace: Kanban, summary, readiness, tests, ambiguity, export hub, analytics, stakeholder preview, command palette, keyboard shortcuts.
 - Bonus Copilot: Natural-language chat over project context (/api/chat, streaming where configured).
 - Bonus estimation: Per-task hours/points/confidence and rollups via effort service and metrics.
 - Bonus export: CSV, Markdown, Jira-oriented and GitHub export paths; human approved_for_export gate.
-

4. Hackathon task mapping (requirements checklist)

1. Requirement ingestion module -- Implemented: POST /api/ingest/text, /file, /url (helix-backend/app/api/routes/ingestion.py). Preprocessing in ingestion_service, ingestion.split_into_clauses, optional store_chunks_mongo, embed_requirements for RAG. UI: New Project tabs (paste, file, URL) plus optional Web Speech dictation into the same textarea (helix-frontend).
 2. Structured outputs -- Implemented: multi-agent pipeline (analyzer, decomposer, etc.) producing stories, tasks, AC, summary, risks; persisted via SQLAlchemy / project graph (helix-backend/app/agents/, project_bridge.py).
 3. Test case generation -- Implemented: TestArchitectAgent, POST /api/testcases/generate/{project_id}, UI lists and status (helix-backend/app/agents/test_architect.py, api/routes/testcases.py).
 4. Ambiguity detection -- Implemented: AmbiguityAgent, POST /api/ambiguity/analyze/{project_id}, NLP supplements in nlp_service.py, UI AmbiguityView.jsx.
 5. Interactive dashboard -- Implemented: Dashboard.jsx and related components (artifacts, tests, export, copilot, onboarding, version history).
 6. Bonus: Conversational assistant -- Implemented: chat routes and CopilotPanel.jsx.
 7. Bonus: Effort estimation -- Implemented: estimator agent, effort_service.py, metrics bar and related API fields.
 8. Bonus: PM export -- Implemented: export routes and ExportHub.jsx; Jira/GitHub require environment configuration documented in SETUP.md and helix-backend/app/config.py.
-

5. Technical architecture

5.1 Logical architecture (ASCII)

```
[ Browser: React SPA ]
  | HTTPS (same origin in all-in-one demo, or /api proxy via Nginx in Compose)
  v
[ FastAPI -- JWT auth, REST, SSE, WebSocket progress ]
  |
  |-- SQLAlchemy -> PostgreSQL (or SQLite demo image)
  |-- Optional Redis -> task progress (in-memory fallback if absent)
  |-- Optional MongoDB -> chunks / requirement snapshots
  |-- In-process FAISS + sentence-transformers (per-project RAG)
  |-- Azure OpenAI / Anthropic / mock agents
```

5.2 Key backend modules

- `app/main.py` -- Application factory, CORS, routers; optional `HELIX_SERVE_SPA` static UI for single-URL demos.
- `app/api/routes/*` -- Auth, projects, ingest, artifacts (incl. streaming), test cases, ambiguity, chat, export, WebSocket progress.
- `app/agents/orchestrator.py` -- Pipeline coordination, metrics, timings.
- `app/services/generation_service.py` -- Background / blocking generation entry points.
- `app/services/ai_service.py` -- Provider abstraction for JSON and streaming.
- `scripts/seed.py` -- Idempotent demo user and seeded project for judges.

5.3 Frontend

- `helix-frontend/` -- Single UI tree (Vite, React). API client uses `VITE_API_BASE` or same-origin `/api`.
- Notable UX: command palette (Ctrl+Shift+P), judge path Load sample requirement on New Project (`docs/RUNBOOK.md`).

5.4 Deployment modes

- Local dev: backend `run.ps1` / `uvicorn` and `npm run dev` -- see `docs/RUNBOOK.md`.
 - Docker Compose: `docker-compose.yml` -- Postgres, Redis, Mongo, backend, Nginx frontend.
 - Public demo (single URL): `Dockerfile.all-in-one` + `render.yaml` -- see `docs/DEMO_HOSTING.md`.
-

6. Data and traceability

- Canonical project graph stored as structured JSON in the database (normalized tables for queries, tests, etc.).
 - Artifacts reference source_clause_ids where applicable to preserve traceability from requirement text to stories, tasks, and tests.
 - ProductivityMetrics exposes narrative fields (e.g. hours saved, coverage, citation_item_rate) for analytics and stakeholder views.
-

7. Security and configuration

- JWT authentication on protected routes; passwords hashed for registered users.
 - Secrets must be supplied via environment variables (see `.env.example`); never committed to the repository.
 - `HELIX_API_KEY` optional gate for hosted demos (`app/api/deps.py`).
 - Sensitive scan on ingest returns non-blocking hints for obvious secret-like patterns in pasted text.
 - CORS configurable via `HELIX_CORS_ORIGINS` for split UI/API deployments; same-origin all-in-one demo avoids extra CORS setup.
-

8. Testing and quality

- Frontend: npm run lint, npm run build in helix-frontend/.
 - E2E: Playwright smoke (helix-frontend/e2e/) uses demo credentials and sample ingest path (documented in docs/RUNBOOK.md).
 - Backend: pytest and agent tests exist under helix-backend/tests/ (run per backend README / CI).
-

9. Known limitations (honest)

- Voice uses browser Web Speech only; quality depends on browser and network.
 - Jira/GitHub export requires correct tokens and project metadata in environment variables.
 - Free PaaS cold starts (e.g. Render free) may add latency after idle periods.
 - SQLite in the all-in-one demo image is suitable for review, not high-concurrency production.
 - LLM quality and cost depend on chosen models and keys; mock mode demonstrates UX without live models.
-

10. Future roadmap (from product narrative)

- Domain-specific prompt packs (e.g. fintech, healthcare).
 - Stronger vector governance and cross-project retrieval policies.
 - Deeper two-way sync with Jira / Azure DevOps and test execution integrations.
-

11. How to run (condensed)

Docker (full stack): From repo root, cp .env.example .env, edit secrets, docker compose up --build. Open UI at <http://localhost:5173> (per SETUP.md).

Local scripts: Backend helix-backend/run.ps1 (port 8765 typical), frontend cd helix-frontend && npm ci && npm run dev (port 5173). Demo user: demo@demo.com / demo123 after seed.

Gold demo path (no microphone): New Project -- Load sample requirement -- Ingest -- Generate artifacts (docs/RUNBOOK.md).

12. File index (quick reference)

- README.md -- Product overview and quick start
 - SETUP.md -- Docker and environment variables
 - docs/RUNBOOK.md -- Canonical ports, judge path, E2E, voice
 - docs/DEMO_HOSTING.md -- Public HTTPS demo via Render
 - ARCHITECTURE.md -- Mermaid diagram and deep architecture
 - docker-compose.yml -- Multi-service local/cloud stack
 - Dockerfile.all-in-one -- Single-service demo image
 - render.yaml -- Render Blueprint definition
 - PRESENTATION.md -- Slide outline; scripts/build_pitch_deck.py builds .pptx
-

13. Sign-off

This report describes the implementation as present in the AIThon repository at the time of writing. For the latest run instructions and submission packaging, prefer docs/RUNBOOK.md and docs/DEMO_HOSTING.md.

Prepared for: Hackathon reviewers and organizers.

Format: Markdown source of truth; PDF generated for portal upload via scripts/build_submission_pdfs.py.